

IronDome VPN Traffic Analyzer

OpenVPN and WireGuard Traffic Detection using Wireshark

Vaibhav Katendra (2024MCS2459)
Nizaul Rahman (2025MCS2099)

1 Introduction

This project aims to analyze VPN traffic by developing a custom Wireshark dissector for OpenVPN and a heuristic detector for WireGuard. The primary goal is to identify VPN traffic independent of port numbers and extract meaningful protocol-level information.

2 Environment Setup

Two Ubuntu virtual machines were configured in VirtualBox:

- **Server VM IP:** 192.168.56.105
- **Client VM IP:** 192.168.56.106

A hybrid networking setup was used:

- NAT adapter (for internet access)
- Host-only adapter (for inter-VM communication)

2.1 IP Configuration

```
server@server-vpn:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:21:02:3b brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.105/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s9
        valid_lft 494sec preferred_lft 494sec
    inet6 fe80::8266:cb3d:4942:2b75/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure 1: Server IP configuration

```

server@client-vpn:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fe:d0:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.107/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s9
        valid_lft 498sec preferred_lft 498sec
    inet6 fe80::a9a9:e6f5:e963:d21c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

Figure 2: Client IP configuration

2.2 Connectivity Test

Basic connectivity was verified using ICMP:

- Client → Server ping successful

```

server@client-vpn:~$ ping 192.168.56.105
PING 192.168.56.105 (192.168.56.105) 56(84) bytes of data.
64 bytes from 192.168.56.105: icmp_seq=1 ttl=64 time=6.69 ms
64 bytes from 192.168.56.105: icmp_seq=2 ttl=64 time=3.57 ms
64 bytes from 192.168.56.105: icmp_seq=3 ttl=64 time=1.41 ms
64 bytes from 192.168.56.105: icmp_seq=4 ttl=64 time=1.32 ms
64 bytes from 192.168.56.105: icmp_seq=5 ttl=64 time=1.10 ms
64 bytes from 192.168.56.105: icmp_seq=6 ttl=64 time=3.13 ms

--- 192.168.56.105 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms

```

Figure 3: Ping from client to server

3 OpenVPN Setup

OpenVPN was configured using Easy-RSA:

- Certificates generated (CA, server, client)
- Server tunnel network: 10.8.0.0/24
- Server VPN IP: 10.8.0.1
- Client VPN IP: 10.8.0.6

```

server@server-vpn: ~
server@server-vpn:~$ echo 'net.ipv4.ip_forward=1' | sudo tee -a /etc/sysctl.conf

sudo sysctl -p
sudo systemctl start openvpn@server
sudo systemctl enable openvpn@server
sudo systemctl status openvpn@server
net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
net.ipv4.ip_forward = 1
net.ipv4.ip_forward = 1
● openvpn@server.service - OpenVPN connection to server
   Loaded: loaded (/usr/lib/systemd/system/openvpn@.service; enabled; preset:
   Active: active (running) since Sat 2026-04-04 05:44:00 IST; 1min 54s ago
     Docs: man:openvpn(8)
           https://community.openvpn.net/openvpn/wiki/Openvpn24ManPage
           https://community.openvpn.net/openvpn/wiki/HOWTO
   Main PID: 1296 (openvpn)
    Status: "Initialization Sequence Completed"
     Tasks: 1 (limit: 10)
  Memory: 2.6M (peak: 2.8M)
     CPU: 43ms
    CGroup: /system.slice/system-openvpn.slice/openvpn@server.service
            └─1296 /usr/sbin/openvpn --daemon ovpn-server --status /run/openvp

```

Figure 4: OpenVPN server active

3.1 Traffic Capture

Traffic was captured on the client interface (enp0s9) using Wireshark.

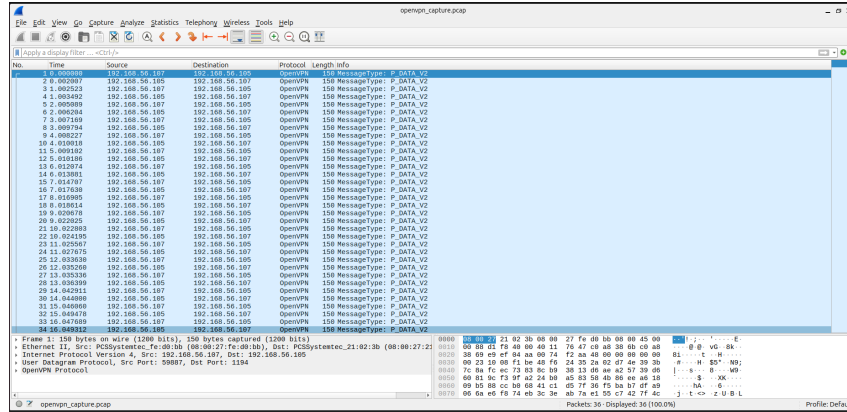


Figure 5: Captured OpenVPN traffic

4 OpenVPN Dissector Design

The dissector was implemented in Lua using heuristic detection.

4.1 Detection Logic

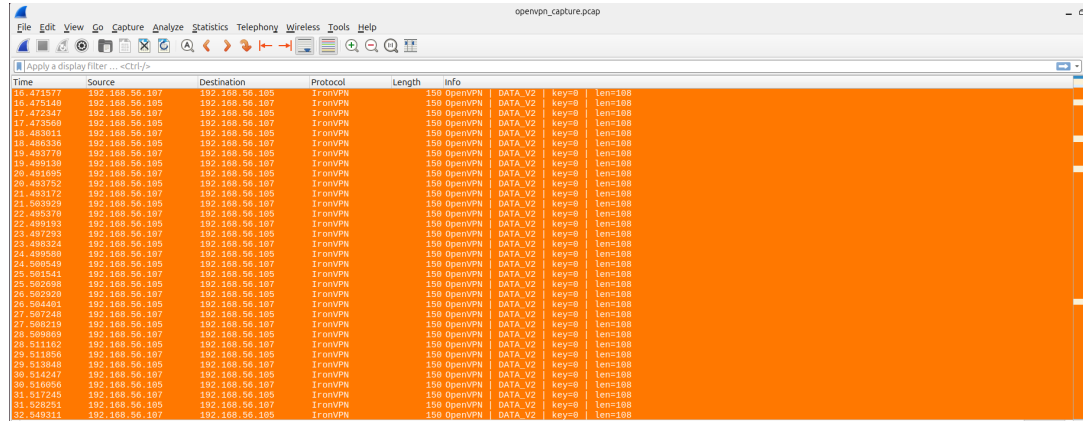
- First byte parsed into opcode (5 bits) and key ID (3 bits)
- Valid opcode range: 1–9
- Control packets validated using session ID
- Packet length checks applied

4.2 Parsed Fields

- Message type (DATA, CONTROL, ACK)

- Key ID
- Session ID
- ACK count and packet IDs
- Payload

4.3 Wireshark Output



Time	Source	Destination	Protocol	Length	Info
17.471577	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
16.475140	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
17.472347	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
17.473560	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
18.483011	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
18.486336	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
19.493776	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
19.499139	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
20.493695	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
20.493752	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
21.493172	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
21.500329	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
22.495370	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
22.499193	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
23.497253	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
23.498324	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
24.499590	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
24.500549	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
25.501541	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
25.502690	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
26.502929	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
26.504401	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
27.507246	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
27.508219	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
28.509899	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
28.511162	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
29.511856	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
29.513846	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
30.514247	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
30.516956	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
31.517245	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
31.520251	192.168.56.105	192.168.56.107	IronVPN	150	OpenVPN DATA_V2 key=0 len=108
32.543311	192.168.56.107	192.168.56.105	IronVPN	150	OpenVPN DATA_V2 key=0 len=108

Figure 6: Custom OpenVPN dissector output

4.4 Disabling Default Protocol

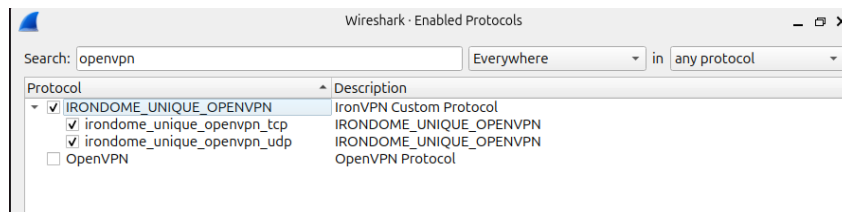


Figure 7: Default OpenVPN protocol disabled

5 Live Traffic Detection

The dissector was tested on live traffic:

- Wireshark capture started
- VPN traffic generated using ping
- Packets dynamically detected and labeled

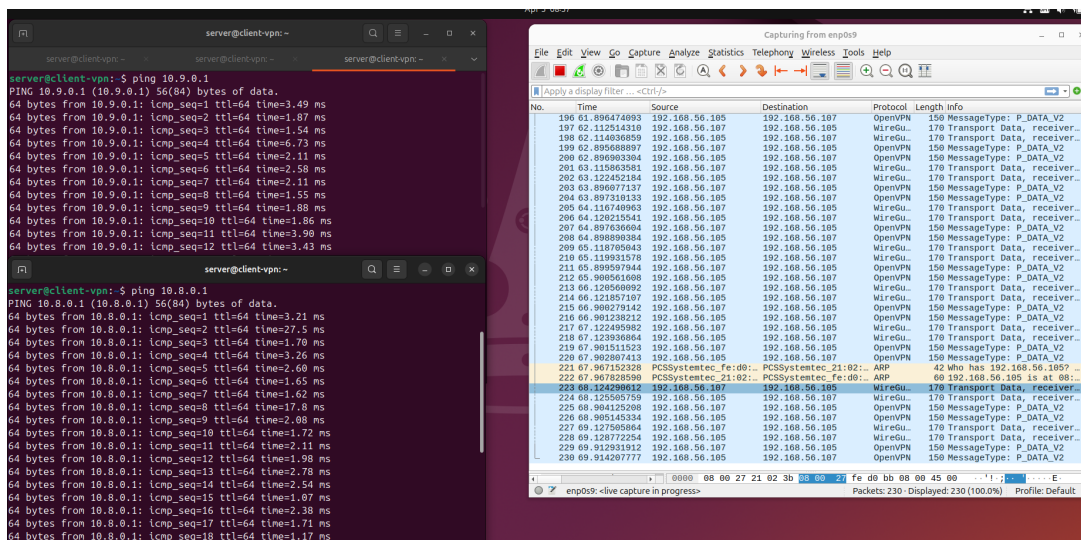


Figure 8: Live traffic detection

6 WireGuard Detection (Bonus)

WireGuard was configured with:

- Server IP: 10.9.0.1
- Client IP: 10.9.0.2

6.1 Detection Logic

- Message types (1–4)
- Reserved bytes must be zero
- Fixed packet sizes for handshake messages

6.2 Wireshark Output

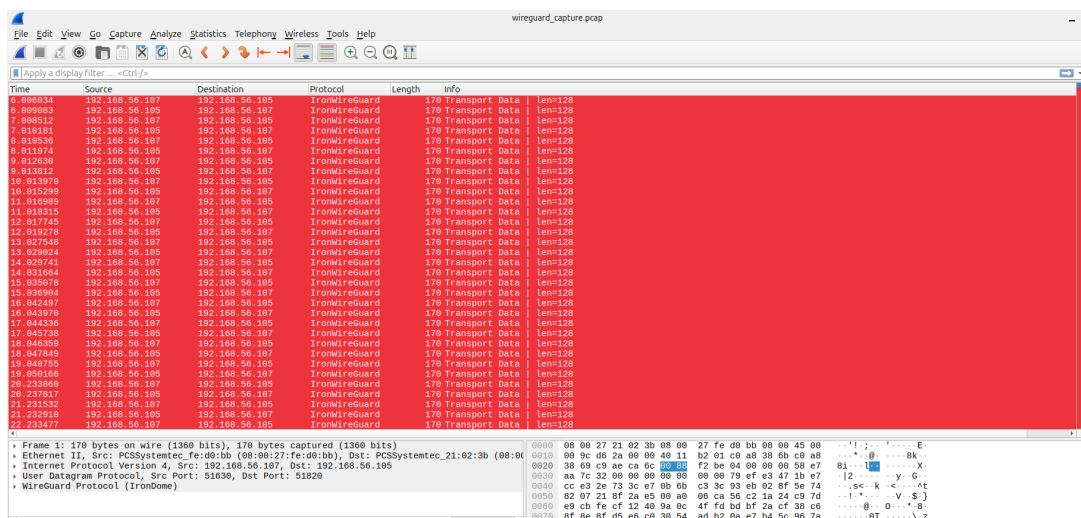


Figure 9: WireGuard detection using custom dissector

7 Challenges

- Bridged networking failed over WiFi; resolved using host-only setup
- Multiple Lua plugin conflicts required cleanup
- Designing accurate heuristics while avoiding false positives
- WireGuard traffic is encrypted, limiting deep inspection

8 Conclusion

The project successfully demonstrates detection and dissection of OpenVPN and WireGuard traffic using custom Wireshark plugins. The heuristic approach ensures port-independent detection while maintaining accuracy. The system works on both live traffic and PCAP files, fulfilling all assignment requirements.